A Survey for LLM Agent Trajectory Analysis: From Failure Attribution to Enhancement

Junjie Wang, Yawen Wang, Mengzhuo Chen, Xiaofei Xie, Chunyang Chen,
Fangwen Mu, Zhe Liu, Qing Wang

Abstract—Large language model (LLM) based agents have emerged as a transformative software paradigm, enabling autonomous task execution through natural language reasoning, multi-agent collaboration, and tool integration. However, their non-deterministic decision-making and complex interaction patterns introduce unprecedented challenges for failure diagnosis and system improvement. Unlike traditional software, where failures manifest as deterministic code defects, agent failures are embedded in lengthy, language-heavy execution trajectories, obscuring root causes and complicating repair. This survey provides a systematic review of the burgeoning field of trajectory analysis for failure attribution and system enhancement in LLM-based agent systems. We collect 42 papers published between early 2025 and February 2026, spanning software engineering, artificial intelligence, and human-computer interaction. We organize the literature along five key dimensions: failure taxonomy, failure attribution, system enhancement and optimization, trajectory monitoring and analysis tools, as well as datasets and benchmarks. Our analysis reveals that while the field has progressed rapidly from simple LLM prompting to causal inference, fine-tuned tracer models, and dynamic intervention, the step-level attribution accuracy remains limited (around 40%), and benchmark diversity is still a bottleneck. We identify four complementary taxonomic perspectives, four methodological paradigms for attribution, and three families of enhancement strategies. We also critically assess existing benchmarks, highlighting a recent shift toward full observability and unrecoverable failure annotation. This survey serves as both a structured overview for current research and a strategic guide for advancing robust, efficient, and self-improving agent ecosystems. We further call upon the software engineering community to engage more deeply with this emerging class of software, bringing expertise in fault localization, program repair, and empirical methods to bear on the unique challenges posed by LLM-based agents.

Index Terms—LLM-based Agent, Failure Attribution, Trajectory Analysis, Agent Operations, AgentOps, Quality Assurance

1 Introduction

With the advancement of large language models (LLMs), we are witnessing a fundamental transformation in the nature of software itself. As Andrej Karpathy conceptualized in his "Software 3.0" framework [1], software is evolving from deterministic, manually coded programs (Software 1.0) and neural network weights (Software 2.0) toward a new era where natural language becomes the primary programming interface and LLMs function as the core reasoning engine. In this new paradigm, prompts effectively act as programs, and systems are built around LLM-based agents (short for LLM Agent) that can reason, plan, and execute tasks autonomously.

On the other hand, software failures are an inevitable aspect of any software product. Consequently, within software engineering research community, failure localization and repair to improve software quality are critical and highly active areas [2], [3]. This holds equally true for LLM agents, where users frequently encounter issues such as unexpected execution breakdown and outputs that deviate from expectations [4]–[6]. Similar to traditional software, repairing these failures to restore correct system behavior requires a non-trivial process of failure attribution, root cause identification, and subsequent fix derivation. For LLM-based agent systems, this process is even more challenging.

Unlike traditional software, where business logic is explicitly encoded in control flow and data structures, the behavior of an LLM-based agent system is largely determined at runtime through model inference, guided by system prompts and inter-agent orchestration. Consequently, failures in such systems are not easily traceable to specific code locations; instead, they manifest in the execution trajectories, i.e., rich and unstructured logs that capture natural language reasoning, multi-step communications, and tool invocations [7], [8]. Moreover, traditional debugging techniques such as breakpoint inspection [9] or deterministic replay [10] are often inapplicable due to the non-deterministic nature of LLM outputs. Thus, **trajectory analysis** becomes the primary means for failure diagnosis, attribution, and repair of LLM agents.

In classical software engineering, similar practices fall under the umbrella of log analysis, which has long been employed for system diagnostics [11]–[14]. However, the language-heavy, non-deterministic, and context-dependent nature of agent trajectories introduces new complexities that challenge traditional log analysis techniques.

First, the inadequacy of structured log analysis for unstructured, non-deterministic trajectories. Traditional

- *J. Wang, Y. Wang, M. Chen, F. Mu, Z. Liu, Q. Wang are with State Key Laboratory of Intelligent Game, Institute of Software Chinese Academy of Sciences, and University of Chinese Academy of Sciences, Beijing, China. J. Wang and Q. Wang are corresponding authors. E-mail: {junjie, yawen2018, liuzhe2020, wq}@iscas.ac.cn*
- *X. Xie is with Singapore Management University, Singapore. E-mail: xfxie@smu.edu.sg*
- *C. Chen is with Technical University of Munich, Munich, Germany E-mail: chunyang.chen@monash.edu*

logs are structured and deterministic, enabling techniques like parsing and anomaly detection [14], [15]. Agent trajectories, however, are unstructured natural language, often varying across runs and containing hallucinated steps, rendering pattern-based methods brittle.

Second, the inability of external observation to capture internal reasoning. Traditional analysis infers faults from external events (e.g., error codes, performance metrics) [16]. Agent failures originate in internal decision processes, requiring techniques that can reconstruct why an agent acted as it did a capability beyond traditional logs.

Third, the mismatch between code-level fixes and system-level repairs. Fault localization in traditional software pinpoints code lines for patching [17]. Agent failures can stem from prompts, workflows, or configurations, where fixes are system-wide. Ineffective localization techniques would offer little guidance for such high-level remediation.

Fourth, the gap between ensuring stability and optimizing capability. Log analysis traditionally focuses on correctness and efficiency. For agents operating in open-ended environments, the goal extends to improving performance in unforeseen scenarios, a shift from ensuring system stability to capability assessment that current methods cannot fully support.

Given that established techniques for failure localization, repair, and log analysis in traditional software can not be directly applicable to LLM agents, a new stream of research has emerged in recent years. This body of work focuses on failure attribution and repair for LLM-based agent systems, leveraging trajectory analysis to enable deep understanding and systematic optimization. In this work, we present a systematic survey of this burgeoning field, i.e., trajectory analysis for failure attribution and system enhancement in LLM-based agent systems.

Research in this direction only began to emerge in 2025, yet within the short span of 14 months from early 2025 to the present, we have already collected 42 relevant papers. This rapid growth indicates the field has garnered significant attention. These papers are published across conferences in software engineering, artificial intelligence, human-computer interaction, and others, reflecting broad, interdisciplinary interest in the challenges of LLM agent failure analysis. Among these collected studies, **failure attribution** and **system enhancement and optimization** constitute the core technical pursuits for diagnosing and improving LLM-based agent systems. The other three, **failure taxonomy**, **trajectory monitoring and analysis tools**, and related **datasets and benchmarks**, provide essential support. As demonstrated in Figure 1, we identify four complementary taxonomic perspectives, four methodological paradigms for attribution, and three families of enhancement strategies. Our analysis also reveals that while the field has progressed rapidly from simple LLM prompting to causal inference, fine-tuned tracer models, and dynamic intervention, the step-level attribution accuracy remains limited (around 40%), and benchmark diversity is still a bottleneck. Beyond summarizing existing work, this survey identifies critical gaps and future directions.

This survey thus serves not only as a structured overview of current research but also as a strategic guide for advancing the reliability and efficiency of LLM-based agent systems. While much of the existing work has originated in the AI community, we hope this survey serves as a clear call for increased engagement from software engineering researchers, who bring deep expertise in fault localization, program repair, and empirical methods to bear on this emerging class of software. Furthermore, by grounding diagnosis in trajectory analysis and linking it to systemic improvement, this field moves closer to realizing robust, self-improving agent ecosystems capable of operating reliably in real-world environments.

2 Background

Execution and Trajectory of LLM-based Agent Systems. We consider an LLM-based agent system $\mathcal{M}$ composed of a finite set of agents $\mathcal{A} = \{a_1, a_2, \ldots, a_N\}$ that collaboratively perform a task τ. The system executes in discrete steps. At each step, a single agent is selected to act, possibly through a dispatcher, router, or planning component that is part of the system. At step t, the acting agent is selected as $a(t) = g(h_{t-1})$, where $g(\cdot)$ represents the system-level agent selection mechanism and h_{t-1} denotes the system context at step $t-1$, summarizing the execution history. The selected agent receives an input $x_t = \phi(h_{t-1})$, and produces an output $y_t = \pi_{a(t)}(x_t)$, where $\pi_{a(t)}$ denotes the policy of agent $a(t)$. The system context is then updated as $h_t = u(h_{t-1}, a(t), x_t, y_t)$. The execution terminates after T steps, producing an execution trace $\mathcal{T} = \langle s_1, s_2, \ldots, s_T \rangle$. Each execution step s_t is associated with a structured observable record $o_t = (step_id_t,\ agent_id_t,\ a(t),\ x_t,\ y_t,\ )$.

We present part of an example execution trajectory below; for a more comprehensive illustration, readers may refer to benchmark studies like [7], [38].

```
{
  "step_id": 1,
  "agent_id": 0,
  "agent_name": "PlannerAgent",
  "input_context": "Task: Plan a three-day Walt
      Disney World visit for a family.",
  "output_content": "I will break the task into
      park selection, ticket options, and visit
      order."

  "step_id": 2,
  "agent_id": 1,
  "agent_name": "SearchAgent",
  "input_context": "Find the main theme parks at
      Walt Disney World.",
  "output_content": "Searching official Disney
      sources.",
  "tool_logs": {
    "tool_name": "web_search",
    "input_arguments": "Walt Disney World theme
        parks",
    "outputs": "Magic Kingdom, EPCOT, Hollywood
        Studios, and Animal Kingdom.",
    "execution_status": "success"
}
```

Task Outcome and Failure Attribution. Given a completed execution trace $\mathcal{T}$, the system produces a task outcome. We define a task outcome function $\Omega(\mathcal{T}) \in \{0, 1\}$, where $\Omega(\mathcal{T}) = 1$ indicates successful task completion and $\Omega(\mathcal{T}) = 0$ indicates failure. Failure attribution aims to identify the agent and the execution

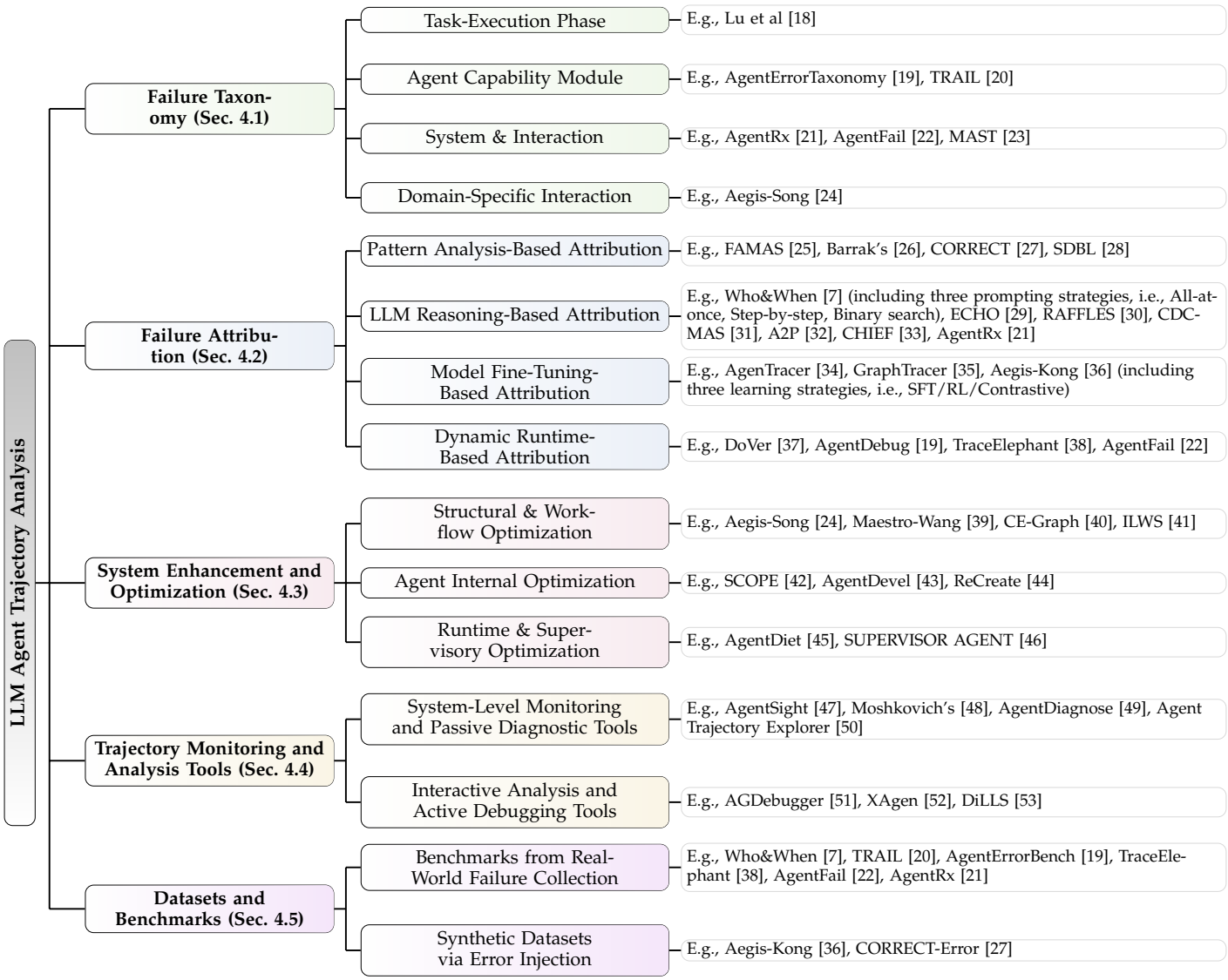

Fig. 1: Overview of related studies about trajectory analysis of LLM-based agent systems.

step responsible for a task failure based on observable execution traces. We define failure attribution at two levels. **Step-level attribution.** Let $\mathcal{T}_{\leq t}$ denote the prefix of the execution trace up to step t. We define the set of feasible continuations from $\mathcal{T}_{\leq t}$ under system $\mathcal{M}$ as $\mathcal{C}(\mathcal{T}_{\leq t}) = \{\mathcal{T}' \mid \mathcal{T}' \text{ is a valid continuation from } \mathcal{T}_{\leq t}\}$. Failure is said to become inevitable at step t if all feasible continuations result in failure: $\forall \mathcal{T}' \in \mathcal{C}(\mathcal{T}_{\leq t}),\ \Omega(\mathcal{T}_{\leq t} \oplus \mathcal{T}') = 0$. The failure step is defined as the earliest inevitable step $t^* = \min\Big\{t \in \{1,\ldots,T\} \mid \forall \mathcal{T}' \in \mathcal{C}(\mathcal{T}_{\leq t}),\ \Omega(\mathcal{T}_{\leq t} \oplus \mathcal{T}') = 0\Big\}$. **Agent-level attribution.** Given an execution trace $\mathcal{T}$ and its failure step t^*, agent-level attribution identifies the agent whose action at the failure step gives rise to the task failure. Formally, the failure-responsible agent is defined as $a^* = \arg\max_{a \in \mathcal{A}} \mathbb{I}\big[\exists\, s_{t^*} \in \mathcal{T} \text{ s.t. } a(t^*) = a\big]$, where $a(t^*)$ denotes the agent selected by the system to act at step t^* in the execution trace $\mathcal{T}$.

System Enhancement and Optimization. Given an agent system $\mathcal{M}$ and a performance objective $\mathcal{J}(\mathcal{M})$ that quantifies its overall capability, such as task success rate, execution efficiency, or robustness in uncertain environments, system enhancement and optimization refers to the process of systematically transforming $\mathcal{M}$ into an improved system $\mathcal{M}'$ by modifying its constituent components (e.g., the agent policies π_a, the selection mechanism $g(\cdot)$, the context-update function $u(\cdot)$, or the input-formation function $\phi(\cdot)$). The goal is to maximize the objective $\mathcal{J}(\mathcal{M}')$ subject to constraints on allowable modifications, often guided by insights derived from the analysis of execution traces $\mathcal{T}_1, \mathcal{T}_2, \ldots$ produced by the original system.

3 Paper Selection and Review Schema

3.1 Paper Collection Methodology

To ensure comprehensive coverage of relevant literature, we employed both automated search (across academic databases) and manual search (within major software engineering and artificial intelligence venues). Our search period spans from January 2023 to November 2025, with a supplementary search conducted for recent publications

from November 2025 to February 2026. It is worth noting that, while our search started from 2023 to avoid potential omissions, all papers ultimately included in this survey were published between 2025 and 2026 as demonstrated in Figure 2.

3.1.1 Automated and Manual Search

To capture research from diverse subfields, we performed an extensive search across four major scientific databases: ACM Digital Library, IEEE Xplore, arXiv, and DBLP.

We initially searched for papers whose titles contained keywords related to LLM agents and trajectory analysis (listed below) in the first three databases. For DBLP, we added keywords related to failure analysis and debugging to filter out irrelevant studies, as relying solely on agent-related keywords would yield an excessively large candidate set. While using two keyword sets for DBLP may omit some studies, we consider this a feasible strategy since many relevant papers from this database are also indexed in the other three databases.

- **Keywords related to both LLM agents and trajectory analysis:** *agent trajectory* OR *LLM agent* OR *multi-agent system* OR *agent failure* OR *agent trace* OR *agent debugging* OR *agent monitoring* OR *agent interaction* OR *agent coordination* OR *agent workflow*.
- **Keywords related to failure analysis and debugging:** *failure attribution* OR *error localization* OR *root cause analysis* OR *debugging* OR *fault diagnosis* OR *trajectory visualization* OR *agent observability* OR *system enhancement*.

We further filtered papers based on their content, retaining only those that mention "LLM", "language model", "agent trajectory", or specific LLM names. This step helps eliminate studies not involving neural or generative agent systems.

To complement automated searches and reduce omission risk, we conducted a manual search within the proceedings of top-tier software engineering conferences and journals, as well as the major artificial intelligence venues considering interdisciplinary nature of LLM agent research.

More than that, after completing the searches and quality assessment, we performed backward snowballing by reviewing the references of collected papers.

3.1.2 Inclusion and Exclusion Criteria

Our search strategy was intentionally broad to maximize initial coverage. To ensure relevance, we defined the following inclusion and exclusion criteria, applied manually to each paper.

Inclusion Criteria:

- The paper proposes or evaluates methods for failure attribution, trajectory analysis, or debugging of LLM-based agent systems.
- The paper introduces datasets, benchmarks, or tools for analyzing or visualizing agent trajectories.
- The paper presents empirical studies or frameworks for enhancing or optimizing LLM agent systems via trajectory analysis.
- The paper focuses on multi-agent coordination, interaction protocols, or system-level observability in LLM agent contexts.

Exclusion Criteria:

- The paper does not involve LLM-based agents (e.g., focuses solely on traditional software or single-model applications).
- The paper uses non-generative or encoder-only models (e.g., BERT) without agentic reasoning or interaction.
- The paper only mentions LLM agents tangentially (e.g., in future work or background sections).
- The paper focuses on testing LLM performance (e.g., fairness, security) rather than agent system behavior.
- The paper is a tool evaluation without proposing analytical or diagnostic methods for agent trajectories.

Two authors independently reviewed each paper against these criteria. Disagreements were resolved by a third author.

3.1.3 Quality Assessment

We further applied a quality assessment checklist to exclude low-quality studies. Each item was scored as "yes" (1), "partial" (0.5), or "no" (0). Papers with a total score below 8 were excluded.

- Is the research goal clearly stated and relevant to LLM agent trajectory analysis or failure diagnosis?
- Is the proposed technique or framework clearly defined and repeatable?
- Does the paper make a clear contribution to the understanding or improvement of LLM agent systems?
- Are the LLM models used explicitly described?
- Is the method of applying LLMs for analysis or attribution explained?
- Is there a clear validation methodology (e.g., datasets, benchmarks, case studies)?
- Are the selected evaluation subjects appropriate for the research goals?
- Are baselines or control methods included to demonstrate effectiveness?
- Are the evaluation metrics relevant to the stated objectives?
- Are the results clearly presented and aligned with the research goals?

3.2 Collection Results

Our initial search retrieved 1,452 papers. After automated filtering, manual screening, application of inclusion/exclusion criteria, and quality assessment, we retained 42 papers relevant to LLM agent trajectory analysis for failure attribution and system enhancement. This significant reduction stems from our deliberately broad initial keyword design, i.e., intended to capture interdisciplinary work, followed by rigorous filtering that removed papers about tangential mentions of agents without substantive contribution to trajectory analysis. Table 1 provides a detailed breakdown of the collected papers.

3.3 General Overview of Collected Papers

Figure 2 illustrates the publication timeline and venue distribution. It can be observed that although LLM-based agents emerged around 2023, research on trajectory analysis began to appear only in 2025 and rapidly grew into a substantial

TABLE 1: Overview of collected studies

ID	Dimension	Paper Title	Approach Name	Date	Venue	Ref.
1	Taxonomy; Benchmark	TRAIL: Trace Reasoning and Agentic Issue Localization	TRAIL	202506	arXiv	[20]
2	Taxonomy; Enhancement	Aegis: Taxonomy and Optimizations for Overcoming Agent-environment Failures in LLM Agents	Aegis-Song	202508	arXiv	[24]
3	Taxonomy	Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks	Lu's	202508	ASE	[18]
4	Taxonomy; Attribution; Benchmark	Where LLM Agents Fail And How They Can Learn From Failures	AgentDebug, AgentErrorTaxonomy, AgentErrorBench	202509	arXiv	[19]
5	Taxonomy	Why Do Multiagent Systems Fail	MAST	202510	Workshop	[23]
6	Taxonomy	How Do LLMs Fail In Agentic Scenarios	Roig's	202512	NeurIPS	[54]
7	Taxonomy; Attribution; Benchmark	AgentRx: Diagnosing AI Agent Failures from Execution Trajectories	AgentRx	202602	arXiv	[21]
8	Taxonomy; Attribution; Benchmark	Demystifying the Lifecycle of Failures in Platform-Orchestrated Agent Workflows	AgentFail	202602	arXiv	[22]
9	Attribution; Benchmark	Which Agent Causes Task Failures and When: On Automated Failure Attribution of LLM Multi-Agent Systems	Who&When	202506	ICML	[7]
10	Attribution	RAFFLES: Reasoning-based Attribution of Faults for LLM Systems	RAFFLES	202508	Workshop	[30]
11	Attribution	Automatic Failure Attribution and Critical Step Prediction Method for Multi-Agent Systems Based on Causal Inference	CDC-MAS	202508	arXiv	[31]
12	Attribution	Who is Introducing the Failure? Automatically Attributing Failures of Multi-Agent Systems via Spectrum Analysis	FAMAS	202508	ESEC/FSE	[25]
13	Attribution; Benchmark	CORRECT: Condensed Error Recognition via Knowledge Transfer in Multi-Agent Systems	CORRECT	202508	arXiv	[27]
14	Attribution	Abduct, Act, Predict: Scaffolding Causal Inference for Automated Failure Attribution in Multi-Agent Systems	A2P	202509	NeurIPS	[32]
15	Attribution	AgentTracer: Who Is Inducing Failure in the LLM Agentic Systems	AgentTracer	202509	ICLR	[34]
16	Attribution	Where Did It All Go Wrong: A Hierarchical Look into Multi-Agent Error Attribution	ECHO	202510	NeurIPS	[29]
17	Attribution	GraphTracer: Graph-Guided Failure Tracing in LLM Agents for Robust Multi-turn Deep Search	GraphTracer	202510	arXiv	[35]
18	Attribution; Benchmark	Aegis: Automated Error Generation and Attribution for Multi-Agent Systems	Aegis-Kong	202510	arXiv	[36]
19	Attribution	Traceability and Accountability in Role-Specialized Multi-Agent LLM Pipelines	Barrak's	202510	arXiv	[26]
20	Attribution	DoVer: Intervention-Driven Auto Debugging for LLM Multi-Agent Systems	DoVer	202512	ICLR	[37]
21	Attribution	From Flat Logs to Causal Graphs Hierarchical Failure Attribution for LLM-based Multi-agent Systems	CHIEF	202601	arXiv	[33]
22	Attribution	Scope Delineation Before Localization A Two-Stage Framework for Enhancing Failure Attribution in Multi-Agent Systems	SDBL	202601	AAAI	[28]
23	Attribution; Benchmark	Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems	TraceElephant	202601	arXiv	[38]
24	Enhancement	Maestro: Joint Graph Config Optimization for Reliable AI Agents	Maestro-Wang	202509	arXiv	[39]
25	Enhancement	Improving the Efficiency of LLM Agent Systems through Trajectory Reduction	AgentDiet	202509	arXiv	[45]
26	Enhancement	Failure-Driven Workflow Refinement	CE-Graph	202510	arXiv	[40]
27	Enhancement	Stop Wasting Your Tokens: Towards Efficient Runtime Multi-Agent Systems	SUPERVISOR AGENT	202510	arXiv	[46]
28	Enhancement	Instruction-Level Weight Shaping: A Framework for Self-Improving AI Agents	ILWS	202512	arXiv	[41]
29	Enhancement	SCOPE: Prompt Evolution for Enhancing Agent Effectiveness	SCOPE	202512	arXiv	[42]
30	Enhancement	AgentDevel: Reframing Self-Evolving LLM Agents as Release Engineering	AgentDevel	202601	arXiv	[43]
31	Enhancement	ReCreate: Reasoning and Creating Domain Agents Driven by Experience	ReCreate	202601	arXiv	[44]
32	Monitoring	Agent Trajectory Explorer: Visualizing and Providing Feedback on Agent Trajectories	Agent Trajectory Explorer	202501*	AAAI	[50]
33	Monitoring	Interactive Debugging and Steering of Multi-Agent AI Systems	AGDebugger	202504*	CHI	[51]
34	Monitoring	AgentSight: System-Level Observability for AI Agents using eBPF	AgentSight	202508	Workshop	[47]
35	Monitoring	Taming Uncertainty via Automation: Observing, Analyzing, and Optimizing Agentic AI Systems	Moshkovich's	202511	arXiv	[48]
36	Monitoring	AgentDiagnose An Open Toolkit for Diagnosing LLM Agent Trajectories	AgentDiagnose	202511*	EMNLP	[49]
37	Monitoring	XAgen An Explainability Tool for Identifying and Correcting Failures in Multi-Agent Workflows	XAgen	202512	CHI	[52]
38	Monitoring	DiLLS: Interactive Diagnosis of LLM-based Multi-agent Systems via Layered Summary of Agent Behaviors	DiLLS	202602	CHI	[53]
39	Others	Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories	Bouzenia's	202510	ASE	[8]
40	Others	MAESTRO: Multi-Agent Evaluation Suite for Testing Reliability and Observability	MAESTRO-Ma	202601	arXiv	[55]
41	Others	Trajectory Guard A Lightweight, Sequence-Aware Model for Real-Time Anomaly Detection in Agentic AI	Trajectory Guard	202601	Workshop	[56]
42	Others	From Features to Actions: Explainability in Traditional and Agentic AI Systems	Chaduvula's	202602	arXiv	[57]

Note: In the Date column, entries marked with an asterisk (*) indicate the publication date of the paper. For all other entries, the date represents when the paper was posted on arXiv, which is typically earlier than the actual publication date. Paper 2 and 18, Paper 24 and 40 have the same method name, and we distinguish them with the surname of first author, e.g., Aegis-Song.

body of work within that same year. Furthermore, with 11 papers already published in just the first two months of 2026, this field continues to develop rapidly.

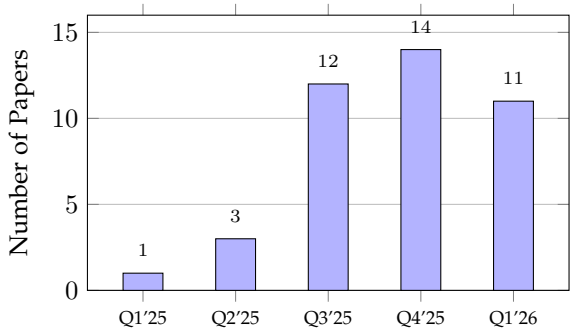

Fig. 2: Quarterly distribution of papers from Q1 2025 to Q1 2026 (Q1 2026 includes only Jan. and Feb.)

As demonstrated in Figure 3, while failure attribution and repair are popular topics in software engineering, papers from software engineering venues currently account for only 7% (3/42) of all publications, lower than the 19% (8/42) from AI venues. Moreover, one of the most influential early works (Who&When) originated from the AI community (ICML 2025 Spotlight). This highlights the interdisciplinary nature of the task, with researchers from multiple fields actively contributing to the topic.

Among the collected publications, while 54% (23/42) currently exist as arXiv preprints. Given the rapid evolution of this field, many significant contributions first appear as preprints before formal publication; we therefore include them after applying our quality assessment checklist (Section 3.1.3) to ensure relevance and technical soundness. This aligns with common practice in survey papers covering emerging topics, where capturing the latest advances often requires incorporating preprint research. While these preprints have not undergone formal peer review, they represent active and often influential contributions to the ongoing discourse.

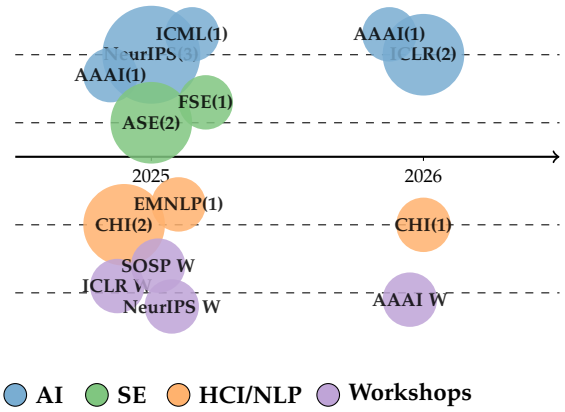

Fig. 3: Timeline and venue distribution of collected studies

4 LLM Agent Trajectory Analysis

We group the related studies in this field along five key dimensions, as illustrated in Figure 4. The research on quality assurance about LLM-based agent systems is organized around two central goals: finding out why failures happen and using that knowledge to build better systems. In our view, the core challenges of the field are **failure attribution** (pinpointing the root cause of the failure within complex interactions) and **system enhancement and optimization** (applying those insights to fix and improve agents). To support this core, three areas provide the necessary foundation. First, **failure taxonomies** offer a common schema to classify failures, which is essential for analyzing and discussing failures systematically. Second, **trajectory monitoring and analysis tools** supply tools needed to observe agent behavior, enabling both automated analysis and human-in-the-loop debugging. Finally, specialized **datasets and benchmarks** are vital for fairly evaluating and comparing different attribution and enhancement methods, ensuring progress is measurable and solid.

Figure 5 illustrates the distribution of research dimensions among the 42 surveyed studies. The data reveals a clear concentration of effort on **failure attribution**, which appears in 18 papers, underscoring its role as the central technical challenge in diagnosing LLM-based agent systems. Following that, **Enhancement** accounts for 8 papers, while **Taxonomy** and **Benchmark** each account for 9 papers, reflecting a balanced focus on both understanding failure modes and improving system performance. There are two papers mainly related with the empirical studies about trajectories of such systems, and we put them into others.

We organize the analysis into subsections, with each subsection focusing on one specific dimension.

4.1 Failure Taxonomy

As mentioned earlier, as an emerging form of software, LLM-based agent systems exhibit failures whose manifestations and characteristics are substantially different from those of traditional software. Consequently, several researchers have sought to analyze why or how these systems fail. They examine the trajectories generated by these systems to diagnose the root causes of failures. Moving beyond individual failure instances, they also develop a structured understanding of why or how failures occur. This effort has led to the creation of many failure taxonomies for such systems.

4.1.1 Categorizing Taxonomies

Researchers have proposed a variety of taxonomies to structure the understanding of failures in LLM-based agent systems. These efforts, while diverse in their specific categorizations, can be synthesized by examining the views through which they analyze system failures. We find that existing work largely converges on four primary analytical perspectives, as shown in Table 2: (1) examining failures through the task-execution phase, (2) attributing them to failures in specific agent capability modules, (3) locating faults within the broader system and interaction landscape, and (4) diagnosing issues inherent to a domain-specific interaction paradigm.

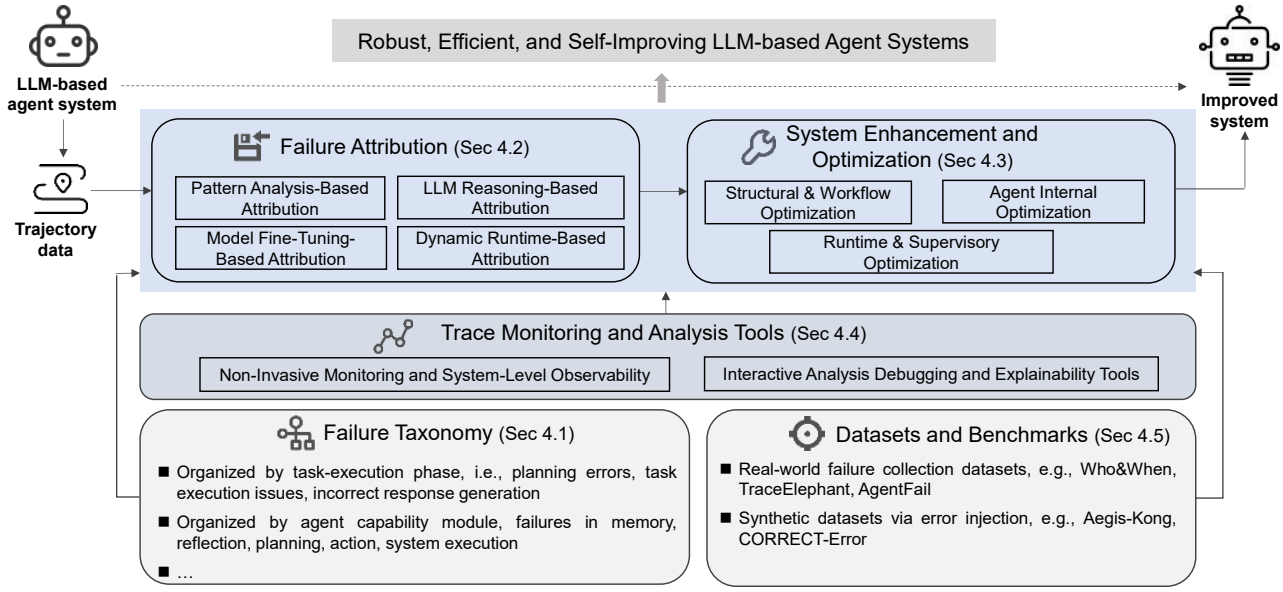

Fig. 4: Overview of five key dimensions in trajectory analysis of LLM-based agent systems

TABLE 2: Categorizing failure taxonomies

Perspective	Core Question	Related Taxonomy	Example L1 Categories
Task-Execution Phase	Where in the task pipeline did it go wrong?	Lu et al. [18]	Task Planning, Task Execution, Response Generation
Agent Capability Module	Which core cognitive function of the agent failed?	AgentErrorTaxonomy [19], TRAIL [20]	Memory, Reflection, Planning, Action, System
System & Interaction	Was the failure in the agent, the coordination, or the platform?	MAST [23], AgentFail [22], AgentRx [21]	Specification, Inter-agent Misalignment, Task Verification
Domain-Specific Interaction	How did the agent fail within this specific paradigm (e.g., environment)?	Aegis-Song [24]	Exploration, Exploitation, Resource Exhaustion

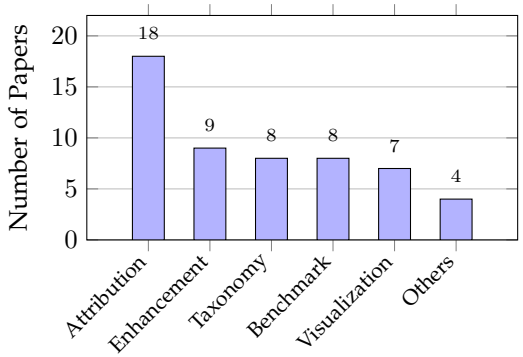

Fig. 5: Distribution of research dimensions in collected studies (note: papers may be assigned multiple topics)

(1) Taxonomies Organized by Task-Execution Phase. This perspective dissects agent failures by following the natural, sequential pipeline of a task. It asks the fundamental debugging question: "Where in the task pipeline did it go wrong?" This perspective is highly intuitive for developers and aligns with operational monitoring, as it directly maps failures to observable phases in a task trajectory.

For example, Lu et al. [18] develop a three-tier taxonomy of failure causes, highlighting planning errors, task execution issues, and incorrect response generation. Specifically, task planning related failures include improper task decomposition, failed self-refinement leading to infinite loops, and unrealistic planning that exceeds agent capabilities. Task execution failures include failure to exploit external tools, production of flawed code, and improper environmental setup. Response generation phase is prone to formatting issues, context window restraint causing information loss, and exceeding the maximum interaction rounds.

(2) Taxonomies Organized by Agent Capability Module. Moving from external workflow to internal cognition, this perspective focuses on the agent's core functional components. It investigates: "Which core cognitive function of the agent failed?" Taxonomies such as AgentErrorTaxonomy [19] and TRAIL [20] adopt this view, organizing failures around modules like memory, reflection, planning, action, and system execution. This perspective is essential for research aimed at enhancing specific agent capabilities and understanding the compositional nature of agent intelligence.

For example, Zhu et al. [19] introduce the AgentErrorTaxonomy, a modular classification of failure modes spanning memory, reflection, planning, action, and system-level operations. Memory failures indicate the errors in recalling or retrieving information (false recall, omission, retrieval failure) that distort subsequent reasoning. Reflection failures are those in monitoring progress or interpreting outcomes (progress misassessment, outcome misinterpretation), blocking course correction. Planning failures include logically unsound or infeasible strategies (impossible actions, constraint ignorance, incoherent

TABLE 3: Overview of failure taxonomy studies

Taxonomy Perspective	Name	Lv.	L1	L1 Category	Total	Top 3 Failures	Dist.	Improv.	Type	Source Systems (Tasks)
Task-Execution Phase	Lu et al. [18]	3	3	Task planning, task execution, response generation	14	Improper task decomposition, incorrect code, max round limits	Partial	Partial	MAS	TaskWeaver, MetaGPT, AutoGen
Agent Capability Module	Agent Error Taxonomy [19]	2	5	Memory, reflection, planning, action, system	17	Inefficient plan, over simplification, progress misjudge	Yes	No	SAS	N/A (ALFWorld, GAIA, WebShop)
	TRAIL [20]	3	3	Reasoning, system execution, planning/coordination	20	Formatting errors, instruction non-compliance, goal deviation	Yes	No	Both	OpenDeepResearch (GAIA), CodeAct (SWE-Bench Lite)
System & Interaction	AgentRx [21]	2	3	Design, coordination, verification	9	Misinterpretation of tool output, instruction adherence, underspecified intent	Yes	No	Both	τ-bench, Flash, Magentic-One
	AgentFail [22]	2	3	Agent, workflow, platform	16	Knowledge / reasoning limitation, poor prompt design, response deviation	Yes	Yes	MAS	10 systems from Dify/Coze (HumanEval, SWE-Bench, TravelPlanner)
	MAST [23]	2	3	Specification issues, inter-agent misalignment, task verification	14	Step repetition, reasoning-action mismatch, fail to ask clarification	Yes	Partial	MAS	MetaGPT, ChatDev, HyperAgent, AppWorld, AG2
Domain-Specific Interaction	Aegis-Song [24]	2	3	Exploration, exploitation, resource exhaustion	6	State-space navigation failure, instruction following failure, tool output processing failure	Yes	Yes	SAS	5 systems (AgentOccam, CRMArena, etc)

Note: Lv., L1, Total, Dist. Improv. are respectively short for taxonomy levels (Lv.), number of categories in the 1st level (L1), total failure types (Total), whether it provides failure distribution (Dist.) and improvement strategies (Improv.). SAS and MAS indicates the type of agent systems, i.e., single agent system (SAS) and multi-agent system (MAS).

subgoals) that cascade into missteps. Action failures are mistakes in executing operations (malformed outputs, incorrect parameters, missing arguments) that are visible but often mask upstream errors. System-level failures are those outside reasoning, such as tool crashes, API mismatches, or exceeding step limits, highlighting system robustness issues.

(3) Taxonomies Organized by System & Interaction. This perspective scales the analysis to the level of agent systems and their ecosystem. It addresses failures emergent from interactions, asking: "Was the failure in the agent, the coordination, or the platform?" Frameworks like MAST [23], AgentFail [22] and AgentRx [21] employ this perspective, with top-level categories like specification issues, inter-agent misalignment, and platform-level failures. This view is critical for diagnosing the complex, often non-deterministic failures that arise in multi-agent collaboration, where the problem lies not in any single node's reasoning but in the communication, shared understanding, or infrastructure governing their collective behavior.

For example, Cemri et al. [23] present MAST (Multi-Agent System Failure Taxonomy), and identify 14 unique failure modes, organized into 3 categories: specification issues, inter-agent misalignment, and task verification. Specifically, specification failures originate from system design decisions, and poor or ambiguous prompt specifications. Inter-agent misalignment failures arise from breakdowns in inter-agent interaction and coordination during execution. Task verification failures involve inadequate verification processes that fail to detect or correct errors, or premature termination of tasks.

(4) Taxonomies Organized by Domain-Specific Interaction. This perspective recognizes that certain deployment paradigms introduce unique challenges and failure modes. It narrows the focus to ask: "How did the agent fail within this specific paradigm (e.g., environment)?" This type of targeted taxonomy is invaluable for achieving robustness in highly specialized domains (e.g., embodied AI, web navigation), where generic failure categories may not capture critical, domain-specific risks.

The Aegis-Song taxonomy [24] is a prime example, structured around failures specific to agent-environment interaction, such as Exploration, Exploitation, and Resource Exhaustion. Specifically, exploration failures occur when agent fails to gather all information required to complete the task; exploitation failures arise when agent incorrectly processes information it has collected, and resource exhaustion accounts for failures due to exceeding the pre-allocated turns/token limit.

4.1.2 Most Prevalent Failures

Table 3 lists the top three failures specified in each failure taxonomy. We consolidate the the most frequently occurring failure types across various studies, as follows.

Instruction & Format Fidelity: This type, manifested as formatting errors (TRAIL), response content deviation (AgentFail), instruction adherence (AgentRx), and instruction non-compliance (TRAIL, Aegis-Song), is

top-ranked. It highlights a persistent struggle to reliably follow output specifications and complex instructions, a fundamental limitation that has also been consistently identified in broader LLM research [58], [59].

Task Planning & Decomposition: Failures like Improper Task Decomposition (Lu et al.), Inefficient Plan (AgentErrorTaxonomy), and Goal Deviation (TRAIL) point to the shortcomings in logically structuring and adhering to long-horizon tasks.

Interaction, State & Coordination: In multi-agent settings, Step Repetition and Reasoning-Action Mismatch (MAST) are prevalent, exposing difficulties in maintaining coherent state and collaboration.

Knowledge & Reasoning Limits: Knowledge or Reasoning Limitation (AgentFail) and Hallucination (implied in many) remain fundamental constraints of the underlying LLMs.

4.2 Failure Attribution

Failure attribution is the cornerstone for diagnosing and improving LLM-based agent systems. It aims at systematically determining which agent is at fault, at which step the critical error occurred, and why. The central challenge lies in reasoning backwards from observed failures within complex, heterogeneous interaction data (spanning LLM reasoning, tool calls, and environmental feedback), to infer the internal logic breakdown and pinpoint the root cause. This task is particularly difficult because the point where a failure is observed is often not its origin. Initial errors in planning, knowledge, or coordination can propagate and transform through multiple steps of agent reasoning and tool execution, obscuring their original source. This propagation creates a significant gap between symptoms and root causes, demanding more sophisticated, inference-based attribution techniques.

Current research on failure attribution for LLM-based agent systems can be broadly organized into four categories, reflecting a progression in analytical depth and intervention intensity. We begin with pattern analysis-based approaches (Section 4.2.1), which passively mine statistical regularities from execution traces to localize faults. Building on observational data, LLM reasoning-based methods (Section 4.2.2) leverage the innate analytical capabilities of foundation models to simulate human-like diagnostic reasoning. To overcome the limitations of general-purpose LLMs, model fine-tuning-based techniques (Section 4.2.3) train specialized tracer models on curated failure datasets, aim at achieving greater efficiency and accuracy. Finally, dynamic runtime-based approaches (Section 4.2.4) adopt an interventionist paradigm, actively testing causal hypotheses through runtime edits and simulations. We introduce each category in detail in the following subsections.

4.2.1 Pattern Analysis-Based Attribution

As the most observational approach among attribution techniques, pattern analysis-based methods localize faults by mining recurring signals across numerous execution traces, relying purely on data-driven statistical analysis without runtime intervention.

FAMAS [25] pioneered this technical line by adapting the concept of spectrum-based fault localization. It operates by systematically replaying and abstracting trajectories to calculate a "suspiciousness" score for each agent action, estimating its likelihood of causing the failure based on its presence in failing versus successful runs. In particular, they propose a novel suspiciousness formula tailored to LLM-based agent systems, which integrates two key factor groups, namely the agent behavior group and the action behavior group, to account for the agent activation patterns and the action activation patterns within the execution trajectories. Barrak et al. [26] formalized a traceable accountability mechanism. In their Planner → Executor → Critic pipeline, blame is assigned by comparing each stage's output to the final answer and ground truth, pinpointing the earliest stage whose unrepaired error persists.

CORRECT [27] advanced the paradigm by focusing on knowledge transfer and real-time efficiency. It distills recurrent agent failures into compact error schemata and reuses them for decisive error recognition in real time.

To enhance efficiency in complex logs, Sun et al. [28] proposed a two-stage process, SDBL (Scope Delineation Before Localization). It first narrows the search space by delineating potential fault regions, drawing on code defect manifestation patterns to inform this scoping, before performing detailed localization within the reduced scope.

4.2.2 LLM Reasoning-Based Attribution

LLM reasoning-based approaches move from mining correlations to simulating human-like diagnostic reasoning. These methods leverage the analytical capabilities of foundation models to infer root causes from complete task trajectories through sophisticated prompting and structured reasoning processes. The earliest instantiation of this paradigm used straightforward prompting over complete trajectories. The pioneering Who&When benchmark [7] established this paradigm by evaluating three basic strategies, i.e., All-at-once, Step-by-step, and Binary search, demonstrating the feasibility but also the limited accuracy of using LLMs on this complex task.

To enhance reliability, subsequent work introduced more structured reasoning processes. ECHO (Error attribution through Contextual Hierarchy and Objective consensus analysis) [29] develops a hierarchical context representation of the entire interaction trace, provides independent objective analyses across these contexts and cross-validating the findings via consensus voting. Zhu et al. present RAFFLES [30], an evaluation architecture that incorporates reasoning and iterative refinement. Specifically, RAFFLES operates as an iterative, multi-component pipeline, using a central Judge to systematically investigate faults and a set of specialized Evaluators to assess not only the systems components but also the quality of the reasoning by the Judge itself, thereby building a history of hypotheses.

The field then evolved to explicitly model causality. CDC-MAS [31] makes two key technical contributions: (1) a performance causal inversion principle, which correctly models performance dependencies by reversing the data flow in execution logs, combined with Shapley values to accurately assign agent-level blame; (2) a novel causal discovery algorithm, that robustly identifies failure steps by tackling the non-stationary nature of interaction data.

TABLE 4: Overview of failure attribution studies in LLM-based agent systems

Category	Approach	Main idea	Additional Data	Base Model	Repair
Pattern Analysis-Based	FAMAS [25]	Spectrum-based analysis	Each test trace generates 20 traces with trajectory replay	Ag.	No
	Barrak's [26]	Planner → Executor → Critic pipeline with a blame function,	No	Ag.	Yes
	CORRECT [27]	Pattern distillation + real-time recognition	Each test trace needs 1 to 10 demonstrations generated by error injection under real-world failure distributions	Ag.	No
	SDBL [28]	Scope delineation leveraging code defect manifestation patterns	No	Ag.	No
LLM Reasoning-Based	Who&When [7]	Three prompting strategies (All-at-once/Step-by-step/Binary search)	No	Ag.	No
	ECHO [29]	Hierarchical context + Consensus voting	No	Ag.	No
	RAFFLES [30]	Reasoning by central judge + Iterative refinement by specialized evaluators	No	Ag.	No
	CDC-MAS [31]	Multi-granularity causal inference by modeling performance dependencies from data flow	No	Ag.	Yes
	A2P [32]	Infer root cause, define corrective intervention, and verify	No	Ag.	No
	CHIEF [33]	Hierarchical causal graph	No	Ag.	No
	AgentRx [21]	Constraint synthesis + LLM judgement	No	Ag.	No
Model Fine-Tuning-Based	AgenTracer [34]	Multi-granular reinforcement learning	2,476 traces generated by programmed error injection	Qwen 3 8B	No
	GraphTracer [35]	Information dependency graphs	2,147 traces generated by graph-aware synthetic data	Qwen 3 8B	No
	Aegis-Kong [36]	Three learning strategies (SFT/RL/Contrastive)	9,533 traces generated by programmed error injection	Llama 3 8B	No
Dynamic Runtime-Based	DoVer [37]	Intervention-driven debugging	No	Ag.	Yes
	AgentDebug [19]	Self-diagnosis + re-rollout	No	Ag.	Yes
	TraceElephant [38]	LLM agent + counterfactual verification	No	Ag.	No
	AgentFail [22]	LLM-based candidate identification + counterfactual verification	No	Ag.	Yes

Note: Repair indicates whether the approach can also repair the failure or enhance the system. Ag. in Base Model column indicates the approach is model agnostic.

DeepScientist et al. [32] propose A2P, which explicitly guides an LLM through a formal three-step reasoning process within a single inference pass: (1) Abduction, to infer the hidden root causes behind an agents actions; (2) Action, to define a minimal corrective intervention; and (3) Prediction, to verify if the intervention resolves the failure.

CHIEF [33] further refined causal analysis by constructing hierarchical causal graphs to trace fault propagation. It first transforms chaotic trajectories into a structured hierarchical causal graph. It then employs hierarchical oracle guided backtracking to efficiently prune the search space via sybthesized virtual oracles. Finally, it implements counterfactual attribution via a progressive causal screening strategy to rigorously distinguish true root causes from propagated symptoms. AgentRx [21] combines constraint synthesis and LLM-based judgement. It synthesizes constraints, evaluates them step-by-step, and produces an auditable validation log of constraint violations with associated evidence, then an LLM-based judge uses this log to localize the critical step and category.

4.2.3 Model Fine-Tuning-Based Attribution

Unlike prompting-based methods, this paradigm focuses on training specialized models dedicated to the failure attribution task. The resulting specialized tracer model learns to efficiently parse complex agent interactions, recognize error propagation patterns, and localize faults with improved accuracy and inference efficiency compared to general-purpose LLM.

AgenTracer [34] established a foundational pipeline for this direction. It first annotates failed agent trajectories via counterfactual replay and programmed fault injection, producing the curated dataset TracerTraj. Leveraging this resource, they develop AgenTracer-8B, a lightweight failure tracer trained with multi-granular reinforcement learning, capable of efficiently diagnosing errors in verbose multi-agent interactions.

GraphTracer [35] moves beyond sequential trajectory analysis to a structured graph representation. It constructs Information Dependency Graphs (IDGs) to explicitly capture how agents reference and build on prior outputs, and to enable tracing fault propagation along dependency edges rather than temporal steps. The training data is produced through graph-aware fault injection, and the failure tracer also trained guided by graph-structural rewards.

Subsequently, Aegis-Kong [36] scaled and systematized this paradigm. It created one of the largest synthetic attribution datasets to date and performed a comprehensive empirical study, directly comparing the efficacy of three distinct learning strategies, i.e., supervised fine-tuning, reinforcement learning, and contrastive Learning, for training robust failure attribution models.

4.2.4 Dynamic Runtime-Based Attribution

Moving beyond passive log analysis, this group of techniques adopts an active, interventionist approach to attribution. The key design principle is to formulate hypotheses about the root cause and then test them through targeted interventions, i.e., systematically testing whether modifying a hypothesized root cause would have averted the failure.

DoVer [37] is an intervention-driven debugging framework, which augments hypothesis generation with active verification through targeted interventions (e.g., editing messages, altering plans). AgentDebug [19] applies a similar idea by assigning fine-grained error types to each step and module, and then guiding targeted re-rollouts based on the diagnosed error types.

The paradigm further evolves to tightly integrate LLM reasoning with dynamic verification. Works like TraceElephant [38] and AgentFail [22] exemplify this by employing an LLM-based candidate identification step followed by counterfactual verification.

4.2.5 Performance Comparison

Figure 6 demonstrates the comparative overview of failure attribution performance in terms of all investigated techniques. Most techniques were evaluated on the Who&When benchmark [7], making their results directly comparable. And the comparison is organized into hand crafted systems and algorithm generated systems following the benchmark's original setup. The three marked with asterisks (*) represent results obtained on the authors' own experimental datasets, which we exclude from further cross-method analysis.

Generally speaking, agent-level accuracy is significantly higher than step-level accuracy, which is well understood because identifying the responsible agent is inherently easier. For algorithm-generated systems, step-level accuracy tends to be higher than for hand-crafted systems, likely because algorithm-generated systems follow more consistent patterns, making it easier to discern underlying regularities, whereas hand-crafted trajectories tend to be longer and more variable, increasing the difficulty of accurate step-level attribution. Furthermore, fine-tuning-based methods achieve relatively better performance, as they leverage additional training data that enables the model to learn the organizational patterns of trajectories and the failure modes. Additionally, some LLM reasoning-based approaches, such as CHIEF and A2P, also deliver competitive results through detailed causal analysis of trajectories.

4.3 System Enhancement and Optimization

The ultimate goal of diagnosing failures is to enable improvement. Once the root cause of a failure is attributed, the next critical step is to leverage this insight to enhance the system's capabilities, reliability, and efficiency. This section focuses on approaches that utilize trajectory analysis to drive the enhancement and optimization. This can be achieved by reshaping the external structure and environment in which agents operate (Section 4.3.1), by innovating the internal drivers and mechanisms of the agents themselves (Section 4.3.2), or by adding a lightweight layer for process management or supervision at runtime (Section 4.3.3).

4.3.1 Structural & Workflow Optimization

This paradigm focuses on enhancing overall performance and robustness by re-designing the collaborative architecture and interactive environment of multi-agent systems from the outside.

Song et al. [24] design Aegis, a set of targeted environment optimizations: 1) environment observability enhancement,2) common computation offloading, and 3) speculative agentic actions. Their optimization focuses exclusively on the agents environment. They do not alter the users task, the agents core logic (i.e., the underlying model and prompting techniques), or the tasks correctness criteria.

As research progressed, the focus expanded from the environment to the collaborative logic itself. Wang et al. [39] introduce Maestro, a framework-agnostic holistic optimizer for that jointly searches over graphs and configurations to maximize agent quality, subject to explicit rollout/-token budgets. Beyond numeric metrics, Maestro leverages reflective textual feedback from traces to prioritize edits, improving sample efficiency and targeting specific failure modes.

In a complementary, more diagnosis-driven approach, CE-Graph [40] first estimated the failure distribution from a pool of counter examples, identified its densest regions as recurring failure modes, and applies targeted, operator-constrained graph edits. ILWS [41] proposes a post-hoc optimization mechanism. It treats system instructions as versioned, editable pseudo-parameters that are evolved by a reflection engine after each session, thereby continuously refining the guiding principles for all agent behavior at the system level.

4.3.2 Agent Internal Optimization

In contrast to modifying external frameworks, this line of research focuses on optimizing agents' internal decision-making logic to enhance their performance.

SCOPE [42] narrows the optimization focus to the agent's core prompts and context strategies. They frame it as an online optimization problem, in which propose a Dual-Stream mechanism that balances tactical specificity (resolving immediate errors) with strategic generality (evolving long-term principles). Furthermore, they introduce perspective-driven exploration to maximize strategy coverage, increasing the likelihood that the agent has the correct strategy for any given task.

AgentDevel [43] formalizes this process as a rigorous release engineering pipeline: first an implementation-blind LLM critic that characterizes failure appearances without accessing agent internals; then script-based executable diagnosis that aggregates dominant symptom patterns and produces auditable engineering specifications; and finally flip-centered gating that prioritizes pass→fail regressions and fail→ pass fixes as first-class evidence.

ReCreate [44] leverages the interaction trajectories to extract concrete signals about the causes of success or failure and the avenues for improvement, enabling the automatic creation and enhancement of domain agents.

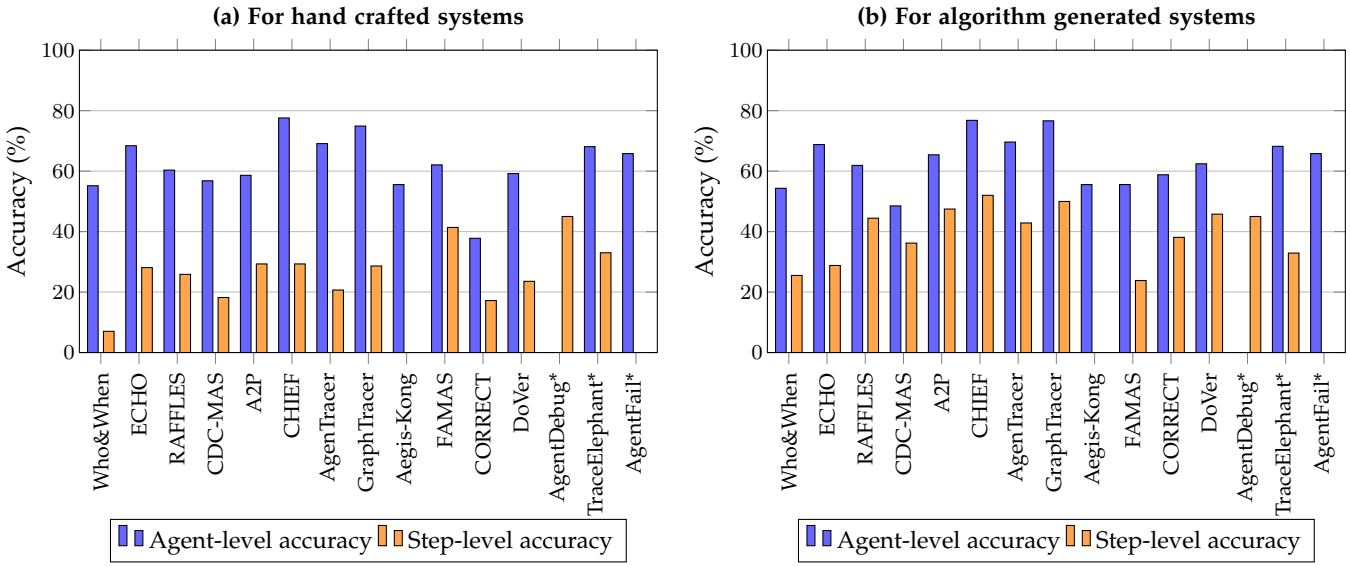

Fig. 6: Failure attribution performance reported in collected studies. Missing bars indicate no reported result for that metric. Methods without asterisks are evaluated on the Who&When benchmark ("With Ground Truth" setting), while asterisks (*) denote evaluations on different test datasets. Approaches not shown in this figure adopt other evaluation metrics.

TABLE 5: Enhancement and Optimization approaches for LLM-based agent systems

Category	Approach	Objec.	Main idea	Timing	Level	What to modify
Structural & Workflow Optimization	Aegis-Song [24]	F + E	Rule-driven environment tuning (enhance observability, offload common computations, speculate agentic actions)	OL	Agent environment interaction	Environment configuration
	Maestro-Wang [39]	F	Graph search + reflective textual feedback	TD	System + Agent	Workflow structure + agent configuration
	CE-Graph [40]	F + E	Graph editing via propose-and-verify based on estimated failure distribution	TD	System	Workflow structure
	ILWS [41]	F + E	Instruction shaping + Feedback learning	OL	System + Agent	Prompts
Agent Internal Optimization	SCOPE [42]	F + A	Prompt evolution + Online learning	OL	Agent	Prompts
	AgentDevel [43]	F	Flip-centered gating with executable diagnostic scripting	TD	Agent	Instructions + Policies
	ReCreate [44]	F + E	Agent-as-optimizer leveraging interaction trajectories	TD	Agent	Instructions + Policies
Runtime & Supervisory Optimization	AgentDiet [45]	E	Rule-driven trajectory compression	OL	Interaction	Interaction data
	SUPERVISOR AGENT [46]	F + E	Proactive error correction, inefficiency guidance, adaptive observation purification	OL	Supervisory layer	External supervision policy

Note: Objec. indicates the optimization objective. F, E, and A in Objec. column are short for Functionality, Efficiency, and Adaptability in new scenarios; OL and TD in Timing column respectively short for Online and Trace-driven.

4.3.3 Runtime & Supervisory Optimization

The third paradigm instead enhances efficiency and robustness by optimizing the data flow of the execution process or attaching a lightweight external supervisory layer.

AgentDiet [45] demonstrated that useless, redundant, and expired information is widespread in all trajectories, which can be identified and reduced without harming the agents performance. They then design a simple yet effective trajectory reduction approach to automatically remove such waste information.

SUPERVISORAGENT [46] introduced a lightweight and modular framework for runtime, adaptive supervision that operates without altering the base agents architecture. Triggered by an LLM-free adaptive filter, it intervenes at critical junctures to proactively correct errors, guide inefficient behaviors, and purify observations.

4.4 Trajectory Monitoring and Analysis Tools

The observability of an agent system, i.e., enabling the capture and understanding of its internal states and interactions, is essential for failure diagnosis and system improvement. This section reviews approaches that enhance observability by making execution traces interpretable, allowing both automated analysis and human-in-the-loop inspection. The works in this area can be broadly grouped by their primary focus: tools for low-level, non-invasive system monitoring and passive diagnosis (Section 4.4.1), versus those for high-level, interactive analysis and active debugging (Section 4.4.2).

4.4.1 System-Level Monitoring and Passive Diagnostic Tools

This category targets at the passive, transparent, and comprehensive collection of execution data from agent systems.

AgentSight [47] is an observability framework which monitors agents from outside their application code at stable system interface, which is used for identifying resource-wasting reasoning loops, revealing hidden coordination bottlenecks. Moshkovich et. al. [48] present the AgentOps automation pipeline, a six-stage process encompassing behavior observation, metric collection, issue detection, root cause analysis, optimized recommendations, and runtime automation. AgentDiagnose [49] provides a framework for quantitative visualization and diagnose. It has (i) an evaluation module that quantifies five core agentic competencies, i.e., backtracking and exploration, task decomposition, observation reading, self-verification, and objective quality, and (ii) a visualization module that highlights trajectory semantics through t-SNE action embeddings, interactive word clouds, and state-transition timelines. Similarly, Agent Trajectory Explorer [50] provides an intuitive interface to support visual inspection and manual annotation of agent execution trajectories.

4.4.2 Interactive Analysis and Active Debugging Tools

Moving beyond data collection, this category focuses on building interactive tools that empower developers to explore and intervene in agent trajectories.

AGDebugger [51] is an interactive debugging tool, which allows users to reset to previous states and modify past agent messages, so as to support iterative, hypothesisdriven debugging. XAgen [52] bridges automated judgment with human-in-the-loop feedback. It can turn execution traces into an interactive flowchart for trajectory visualization, and also allows users to annotate outputs which serves as feedback for failure attribution and iterative improvement. DiLLS [53] is designed to reveal and structure the behaviors of multi-agent systems, and it organizes execution information into three layers, i.e., overall planning and execution, action arrangement and outcomes, and logs for concrete operations.

4.5 Datasets and Benchmarks

Benchmarks and datasets provide foundational support for researches in this field: they serve as objective, unified evaluation standards for measuring and comparing the performance of different attribution techniques, and as sources of high-quality training data for powering data-driven diagnostic models. Existing studies can be broadly categorized into two types: evaluation-centric benchmarks collected from real-world failures, and synthetic, error-injected datasets designed for both training and evaluation.

4.5.1 Benchmarks from Real-World Failure Collection

These datasets collect naturally occurring failures from actually deployed agent systems, thus closely reflecting real-world application scenarios.

Zhang [7] introduce the Who&When benchmark, the pioneering benchmark in this field, being utilized by almost all failure attribution studies. It collected 127 failure trajectories collected from two LLMs-based multi-agent systems, including both algorithmically generated and hand-crafted systems. Each failure log is accompanied by meticulous annotations, specifying the failure-responsible agent, the corresponding failure step, and some with the reasons for failure in natural language. Similarly, TRAIL [20] and Agent Error Benchmark [19] provides finer-grained root cause, to facilitate the better understanding of the failures.

Chen et al. [38] find existing benchmarks rely on partially observable traces that capture only agent outputs, omitting the inputs and context that developers actually use when debugging. They argue that failure attribution should be studied under full execution observability, aligning with real-world developer facing scenarios where complete traces, rather than only outputs, are accessible for diagnosis. To this end, they introduce TraceElephant [38], a benchmark designed for failure attribution with full execution traces and reproducible environments.

AgentFail [22] collect failure trajectories from two agentic workflow development platforms. Its key contribution is providing fine-grained, lifecycle-spanning annotations for each failure instance, including the precise location, a root cause description categorized according to their developed taxonomy, and a corresponding repair strategy, which enables deeper analysis of the entire failure process beyond simple error detection.

Barke et al. argues that agents can recover from steps labeled as failures in the Who&When benchmark, thereby they introduce AgentRx [21] benchmark that targets the unrecoverable failure step for more meaningful attribution. Besides the execution trajectory, the benchmark also contains a brief natural-language description, and a failure label from their own failure taxonomy with supporting rationale.

4.5.2 Synthetic Datasets via Error Injection

These datasets are constructed by actively injecting errors into successful trajectories, resulting in precise error localization and annotation. Besides evaluation, they are also suitable for model training.

Kong et al. [36] construct Aegis, a large dataset of 9,533 trajectories with annotated faulty agents and error modes. This is achieved using a LLM-based manipulator that can adaptively inject context-aware errors into successful execution trajectories. Yu et al. [27] construct CORRECT-Error, a large-scale collection of over 2,000 multi-agent trajectories with fine-grained, step-level error annotations. It is built using a error-injection pipeline guided by real-world failure distributions, yielding trajectories that balance controlled coverage with the realism of natural MAS failures.

5 Challenges and Opportunities

Based on the above analysis for trajectory related studies, we summarize the challenges and opportunities for future research.

5.1 Improving Attribution Accuracy via Enhanced Causal Understanding

Precise failure attribution remains a significant challenge due to the non-deterministic reasoning and intricate interactions inherent to LLM-based agent systems, with step-level accuracy on established Who&When benchmarks still

TABLE 6: Datasets and Benchmarks for Failure Analysis

Data Type	Benchmark	Source Systems	Scale	Annotation	Adoption	Usage	Specialty
RW	Who&When [7]	CaptainAgent (GAIA, AssistantBench), Magnetic-One (GAIA, AssistantBench)	127	Responsible agent, faulty step, some with root cause	High	Eval.	First benchmark with clear data organization
	TRAIL [20]	OpenDeepResearch (GAIA), CodeAct agent (SWE-Bench Lite)	148	Coarse and fine grained root cause (error type, location)	Low	Eval.	
	AgentError Bench [19]	N/A (ALFWorld, GAIA, WebShop)	200	Failure step, coarse and fine grained root cause (module and error type)	Low	Eval.	
	TraceElephant [38]	CaptainAgent (GAIA, AssistantBench), Magnetic-One (GAIA, AssistantBench), SWE-Agent (SWE-Bench verified)	220	Responsible agent, faulty step	Low	Eval.	Full observable data and running environment
	AgentFail [22]	10 systems from Dify and Coze platform (e.g., HumanEval, SWE-Bench, TravelPlanner)	307	Responsible agent, root cause, repair strategy	Low	Eval.	
	AgentRx [21]	τ-bench, Flash, Magentic-One	115	Failure step, root cause (failure label), first unrecoverable failure	Low	Eval.	Marks the first unrecoverable failure
EI	Aegis-Kong [36]	6 systems (Magnetic-One, MacNet, DyLAN) from 6 task sources (GAIA, MATH, HumanEval)	9,533	Responsible agent, root cause (error mode)	Low	Trn./ Eval.	
	CORRECT-Error [27]	Magnetic-One and AutoGen (GAIA, Math500, MMLU-Pro, HotpotQA)	2,000+	Faulty step	Low	Trn./ Eval.	

Note: RW and EI refer to Real-World failures and synthetic data by Error Injection respectively; Eval. and Trn. refer to Evaluation and Training. Scale indicates the number of failure trajectories. Adoption indicates the extent of usage by other studies.

limited (e.g., about 40% as shown in Figure 6). The difficulty lies in moving beyond surface-level trajectory inspection to the structured understanding of the agent's internal decision logic. CHIEF [33], which achieves the highest performance so far, begin to address this by transforming flat execution logs into hierarchical causal graphs, explicitly modeling data transitions and error propagation. This demonstrates that formalizing the latent logical and causal relationships within trajectories is a viable path to improved attribution.

This represents a valuable initial step in this research direction. Future exploration can develop more comprehensive models that can capture both the deterministic elements of an agent's architecture (e.g., workflow structure, tool call protocols) and the probabilistic nature of its LLM-based reasoning. Furthermore, liking traditional software usually benefits from the combination of static analysis and dynamic running [60], [61], paradigm-breaking hybrid techniques might be another promising direction.

5.2 Rethinking Evaluation for LLM Agent Failure Analysis

The evaluation of failure attribution techniques must align with their intended use in debug and repair. We identify three key dimensions where current practices fall short.

5.2.1 From Accuracy to Utility: The Need for Repair-Driven Metrics

The evaluation of failure attribution techniques currently centers on agent-level and step-level accuracy. However, this evaluation may not fully capture a technique's practical utility in enabling system repair. In traditional software, repairing a defect typically requires precise localization to the faulty code line for targeted modification [62]. In contrast, fixes for LLM-based agent systems often involve system-level changes, such as prompt optimization, workflow restructuring, or agent configuration adjustments, which might not necessarily require knowing the exact failing step. This raises a critical question: does automated repair of agent systems truly demand step-level attribution accuracy?

Consequently, there is an opportunity to design evaluation metrics that are more aligned with the repair, which is in fact the ultimate objective of failure analysis, rather than merely accurate attribution. New metrics could assess how well an attribution method facilitates actionable repairs or directly contributes to measurable system enhancement. On the other hand, Ma et al. [22] also pointed that descriptions of root causes directly aid in repair. Therefore, when attributing a failure to a specific location, it is also highly valuable to provide a description of why the failure occurred. Currently, only a few studies can directly offer such information.

5.2.2 Beyond Partial Observability: The Demand for Comprehensive Benchmarks

Who&When [7] is almost the only benchmark specifically designed for failure attribution of LLM-based agent systems for a considerable period in the past. It provides partially observable execution traces that include only the agents outputs, without the corresponding inputs such as original task instructions, prompts, or contextual messages. This restricted setting can be suitable for evaluating attribution methods under certain black-box scenarios, where internal inputs or intermediate states are inaccessible (e.g., debugging deployed agents on external platforms). However, in many practical developer-facing debugging scenarios, developers typically have access to substantially

richer execution information, including task instructions, prompts, intermediate messages, tool invocations, and environment states.

The recent introduction of benchmarks like TraceElephant [38], which offers complete execution traces and reproducible environments, addresses this gap and represents a necessary step forward. Future opportunities include creating more diverse, large-scale benchmarks from real-world deployments to robustly evaluate both attribution and enhancement methods under realistic conditions.

5.2.3 From Binary Outcomes to Capability Assessment: Evaluating in Open-Ended Scenarios

Most existing failure diagnostic methods provide only coarse-grained outputs, such as binary failure labels, and heavily rely on the availability of task-specific oracles (e.g., test cases) to determine success. However, LLM-based agents are increasingly deployed in open-ended scenarios (e.g., creative tasks) where clear oracles are scarce, moving beyond simple pass/fail judgments toward assessing agent capabilities in open-ended scenarios.

An initial step in this direction is demonstrated by AgentDiagnose [49], which provides a preliminary attempt to evaluate agents across five competency dimensions using LLM-as-a-judge. A critical opportunity lies in developing trajectory-based analysis that delivers fine-grained, multidimensional assessments of agent capabilities (e.g., planning fidelity, tool usage, state tracking). Such detailed diagnostics would not only aid developers in pinpointing weaknesses but could also serve as intermediate supervision signals to improve agent training.

5.3 Toward Systematic AgentOps: From Fragmented Tools to Integrated Governance

Despite progress in attribution, enhancement, and monitoring, these capabilities remain fragmented, i.e., there is no integrated framework to govern LLM agents from offline evaluation to production operation. Emerging industrial initiatives such as AgentOps SDK [63] and IBM Telemetry [64] signal the need for a systematic discipline, yet academic research lags behind. As pointed by a recent survey [65], AgentOps typically includes four key stages: monitoring, anomaly detection, root cause analysis, and resolution, together contributing to the accountable, observable agent systems.

To achieve this, we need open observability standards, attribution methods that work online, benchmarks that reflect real-world conditions, and governance frameworks that keep humans in the loop. Progress has been made on individual fronts, but an integrated, systemlevel solution remains absent. We believe AgentOps should be a firstclass research focus, not a tooling afterthought, if we are to move from brittle prototypes to reliable production systems.

5.4 Expanding the Horizon: Trajectory Analysis in the Wild

Beyond controlled settings, two directions connect trajectory analysis to real-world failure diagnose.

5.4.1 Learning from Production Deployments

Current benchmarks and datasets are predominantly constructed from controlled environments or synthetic error injection. While valuable for establishing baselines, they may not capture the full diversity and unpredictability of failures encountered in real-world deployed agent systems. Production environments introduce complexities that controlled settings cannot replicate: ambiguous user requests, evolving external APIs, fluctuating latency, and unanticipated edge cases [66]. Moreover, failures in deployment often intertwine with system-level issues (e.g., resource constraints, network instability) that are absent from laboratory evaluations.

This gap presents a potential opportunity for empirical software engineering research, e.g., for large-scale, longitudinal studies that collect trajectory data from production environments, documenting not only failures but also successful recoveries, adaptation patterns, and the long-term effects of system updates. Such empirical foundations would enable more realistic evaluation of attribution and repair techniques, and inspire methods tailored to practical constraints for online diagnosis.

5.4.2 Domain-Specific Failure Diagnose

Different application domains impose unique requirements and introduce distinct failure modes that generic trajectory analysis methods may overlook. Consider coding agents (e.g., SWE-agent [67], AutoCodeRover [68]), which navigate large codebases, generate patches, and interact with version control systems. Their trajectories contain rich structural informationcode snippets, dependency graphs, test outputsthat generic natural language methods fail to exploit. Similarly, GUI agents [69] interact with dynamic visual interfaces, interpreting screen layouts, handling UI events, and adapting to application state changes. Their failures often stem from misinterpreting visual elements, incorrect action sequences, or inability to handle unexpected interface transitions.

Other domains, such as web navigation agents and embodied agents in robotics, each exhibit characteristic failure patterns. Developing specialized diagnostic tools that incorporate domain knowledge (e.g., code structure, screen layouts, dialogue acts) could significantly improve the accuracy and actionability of attribution methods. Moreover, cross-domain transfer learning might reveal generalizable principles of agent failure, informing the design of more robust diagnose techniques.

6 Related Work

6.1 LLM Agents for Software Engineering Tasks

The software engineering community has rapidly embraced LLMbased agents to automate a wide spectrum of development activities. Multiagent frameworks such as ChatDev [70] and MetaGPT [71] simulate collaborative software teams that write code, conduct code reviews, and manage projects from highlevel specifications. Specialized agents like SWEagent [67], AutoCodeRover [68] and ExpeRepair [72] tackle realworld GitHub issues by navigating repositories, reproducing bugs, and generating

patches, achieving stateoftheart results on benchmarks such as SWEBench. Beyond coding, agents have been deployed for unit test case generation [73], fault localization [74], and automated software testing [75], demonstrating their potential to reshape software engineering workflows.

However, the adoption of these agents introduces new reliability challenges as demonstrated in Section 1. Bouzenia and Pradel [8] conducted the first empirical study of thoughtactionresult trajectories of software engineering agents, revealing recurrent failure modes such as repeated identical actions without follow-up, and debugging antipatterns, such as generating a fix without testing it. Their work highlights a pressing need for systematic, automated techniques to diagnose why an agent fails and how to repair its behavior. This survey directly addresses that need.

6.2 Automatic Construction and Self-Evolution of LLM Agents

Parallel to the software engineering communitys focus on deploying agents for development tasks, the artificial intelligence community has intensively explored how to automatically construct, optimize, and evolve LLMbased agents from scratch or from minimal seeds.

Early work framed agent customization as prompt tuning or incontext learning, but recent advances treat the entire agent its instructions, policies, tool use patterns, and even collaborative workflows as optimizable parameters. For example, SCOPE [42] formulates prompt evolution as an online learning problem, balancing tactical error resolution with strategic principle refinement via a dualstream mechanism and perspectivedriven exploration. Another line of research focuses on optimizing the multiagent collaboration structure through systematic exploration of the design space, driven by diagnostic signals extracted from trajectories.. For example, Maestro [39] jointly searches over graph topologies and agent configurations under token or rollout budgets, guided by reflective textual feedback from traces.

Despite their diversity, all these automatic construction techniques depend critically on understanding why an agent succeeds or fails precisely the focus of this survey.

6.3 AgentOps: Observability and Operational Management for LLM-based Agent Systems

Parallel to the technical advances in failure attribution and system enhancement surveyed in this paper, a complementary discipline has emerged at the intersection of software engineering, DevOps, and AI infrastructure: AgentOps. While our work focuses more on how to analyze trajectories to diagnose and repair failures, AgentOps addresses the broader question of how to observe, monitor, and manage agent systems throughout their entire production lifecycle, such as AgentOps SDK [63], IBM Telemetry with watsonx Orchestrate [64].

Dong et. al. [76] propose the systematic taxonomy of AgentOps, identifying traceable artifacts across the agent production lifecycle and emphasizing observability as a prerequisite for reliability. Moshkovich and Zeltyn [48] articulate a six-stage AgentOps automation pipeline that explicitly integrates root cause analysis and optimization recommendation. Wang et. al. [65] conducts a survey, in which they provide detailed definitions and explanations of the four key stages of AgentOps: monitoring, anomaly detection, root cause analysis, and resolution. We can see that, they all have stages that correspond to the failure attribution and system enhancement dimensions central to our survey.

Existing AgentOps literature are mainly industry white papers, platform documentation, and early-stage academic taxonomies. And they focus on observability infrastructure, platform architecture, and operational best practices rather than the analysis of attribution approaches or enhancement strategies. Our work complements this emerging discipline by offering the first structured roadmap of the diagnostic techniques that AgentOps platforms can (and increasingly do) operationalize.

7 Conclusion

Trajectory analysis has rapidly emerged as the central lens through which LLMbased agent systems can be understood, diagnosed, and improved. This survey has provided a systematic overview of the emerging field of trajectory analysis for failure attribution and system enhancement in LLM-based agent systems. By collecting and synthesizing 42 relevant papers published between early 2025 and February 2026, we have mapped the research landscape along five key dimensions: failure taxonomy, failure attribution, system enhancement and optimization, trajectory visualization and analysis tools, as well as datasets and benchmarks.

Our analysis reveals a field in rapid formation, driven by the recognition that traditional software diagnosis techniques are illsuited for the nondeterministic, languageheavy, and interactionintrinsic nature of agent failures. By highlighting both achievements and persistent gaps, we aim to guide researchers and practitioners toward the next generation of robust, accountable, and self-improving agent ecosystems. The journey from todays brittle prototypes to reliable, deployable agentic systems will depend critically on our ability to analyze, attribute, and act upon the rich information embedded in every agent trajectory. We hope this work accelerates that journey.

References

[1] A. Karpathy, "State of GPT," Talk at Microsoft Build, 2023. [Online]. Available: https://www.youtube.com/watch?v=bZQun8Y4L2A

[2] Y. Lou, A. Ghanbari, X. Li, L. Zhang, H. Zhang, D. Hao, and L. Zhang, "Can automated program repair refine fault localization? a unified debugging approach," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 75–87.

[3] M. Wen, J. Chen, Y. Tian, R. Wu, D. Hao, S. Han, and S.-C. Cheung, "Historical spectrum based fault localization," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2348–2368, 2019.

[4] R. Ramakrishnan, "Coding agents don't need to be perfect, they need to recover: Error analysis of 8 models on agentic coding tasks," Snorkel AI Blog, Feb. 2026. [Online]. Available: https://snorkel.ai/blog/coding-agents-dont-need-to-be-perfect-they-need-to-recover/

[5] R. Ramesh, "Ai boss fails spectacularly in month-long business test: Anthropic claude agent loses money, hoards tungsten, believes it's human," AIToday, Jun. 2025. [Online]. Available: https://www.aitoday.io/ai-boss-fails-spectacularly-in-month-long-business-test-a-28862

[6] Real Story Group, "From agentic pilot to production, part 2: Disobedient or just probabilistic?" Real Story Group Blog, Feb. 2026. [Online]. Available: https://realstorygroup.com/Blog/agentic-pilot-production-part-2-disobedient-or-just-probabilistic

[7] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen *et al.*, "Which Agent causes task failures and when? on automated failure attribution of LLM Multi-Agent systems," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 1–12, spotlight paper.

[8] I. Bouzenia and M. Pradel, "Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories," in *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 1–12.

[9] C. Zhang, J. Yang, D. Yan, S. Yang, and Y. Chen, "Automated breakpoint generation for debugging." *J. Softw.*, vol. 8, no. 3, pp. 603–616, 2013.

[10] Y. Chen, S. Zhang, Q. Guo, L. Li, R. Wu, and T. Chen, "Deterministic replay: A survey," *ACM Computing Surveys (CSUR)*, vol. 48, no. 2, pp. 1–47, 2015.

[11] J. Xu, R. Yang, Y. Huo, C. Zhang, and P. He, "Divlog: Log parsing with prompt enhanced in-context learning," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.

[12] D. Yuan, S. Park, and Y. Zhou, "Characterizing logging practices in open-source software," in *2012 34th international conference on software engineering (ICSE)*. IEEE, 2012, pp. 102–112.

[13] D. Yuan, J. Zheng, S. Park, Y. Zhou, and S. Savage, "Improving software diagnosability via log enhancement," *ACM Transactions on Computer Systems (TOCS)*, vol. 30, no. 1, pp. 1–28, 2012.

[14] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 492–504.

[15] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *2017 IEEE international conference on web services (ICWS)*. IEEE, 2017, pp. 33–40.

[16] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "An evaluation study on log parsing and its use in log mining," in *2016 46th annual IEEE/IFIP international conference on dependable systems and networks (DSN)*. IEEE, 2016, pp. 654–661.

[17] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 707–740, 2016.

[18] R. Lu, Y. Li, and Y. Huo, "Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks," in *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 1–12, new Ideas Track.

[19] K. Zhu, Z. Liu, B. Li, M. Tian, Y. Yang, J. Zhang, P. Han, Q. Xie, F. Cui, W. Zhang, X. Ma, X. Yu, G. Ramesh, J. Wu, Z. Liu, P. Lu, J. Zou, and J. You, "Where LLM Agents Fail and How They can Learn From Failures," 2025. [Online]. Available: https://arxiv.org/abs/2509.25370

[20] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian, "TRAIL: Trace Reasoning and Agentic Issue Localization," 2025. [Online]. Available: https://arxiv.org/abs/2505.08638

[21] S. Barke, A. Goyal, A. Khare, A. Singh, S. Nath, and C. Bansal, "Agentrx: Diagnosing ai agent failures from execution trajectories," 2026. [Online]. Available: https://arxiv.org/abs/2602.02475

[22] X. Ma, X. Xie, Y. Wang, J. Wang, B. Wu, M. Li, and Q. Wang, "Demystifying the lifecycle of failures in platform-orchestrated agenticworkflows," 2026.

[23] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why Do Multi-Agent LLM Systems Fail?" in *Proceedings of the 2nd Workshop on Building Trust in Machine Learning at ICLR 2025*, 2025, pp. 1–15.

[24] K. Song, A. Jayarajan, Y. Ding, Q. Su, Z. Zhu, S. Liu, and G. Pekhimenko, "Aegis: Taxonomy and Optimizations for Overcoming Agent-Environment Failures in LLM Agents," 2025. [Online]. Available: https://arxiv.org/abs/2508.19504

[25] Y. Ge, L. Xie, Z. Li, Y. Pei, and T. Zhang, "Who is Introducing the Failure? Automatically Attributing Failures of Multi-Agent Systems via Spectrum Analysis," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2026, pp. 1–12.

[26] A. Barrak, "Traceability and Accountability in Role-Specialized Multi-Agent LLM Pipelines," 2025. [Online]. Available: https://arxiv.org/abs/2510.07614

[27] Y. Yu, M. Li, S. Xu, J. Fu, X. Hou, F. Lai, and B. Wang, "CORRECT: Condensed error Recognition via Knowledge Transfer in Multi-Agent Systems," 2025. [Online]. Available: https://arxiv.org/abs/2509.24088

[28] K. Sun, W. Li, B. Dong, Y. Lin, J. Zhang, and B. Shi, "Scope Delineation Before Localization: A Two-Stage Framework for Enhancing Failure Attribution in Multi-Agent Systems," in *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI)*, 2026, pp. 1–10.

[29] A. Banerjee, A. Nair, and T. Borogovac, "Where Did It All Go Wrong? A Hierarchical Look into Multi-Agent Error Attribution," in *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS)*, 2025, pp. 1–15.

[30] C. Zhu, S. Hong, J. Wu, K. Chawla, C. Tang, Y. Yin, N. Wolfe, E. Babinsky, and D. Liu, "RAFFLES: Reasoning-based Attribution of Faults for LLM Systems," in *Proceedings of the 1st Workshop on Multi-Task and In-Context Learning with Large Language Models (MTI-LLM) at NeurIPS 2025*, 2025, pp. 1–12.

[31] G. Ma, J. Zhu, H. Guo, W. Shi, J. Shen, J. Liu, and Y. Liang, "Automatic Failure Attribution and Critical Step Prediction Method for Multi-Agent Systems Based on Causal Inference," 2025. [Online]. Available: https://arxiv.org/abs/2509.08682

[32] A. West, Y. Weng, M. Zhu, Z. Lin, Z. Ning, and Y. Zhang, "Abduct, Act, Predict: Scaffolding Causal Inference for Automated Failure Attribution in Multi-Agent Systems," in *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS)*, 2025, pp. 1–15.

[33] Y. Wang, W. Wu, J. Wang, and Q. Wang, "From flat logs to causal graphs: Hierarchical failure attribution for llm-based multi-agent systems," 2026.

[34] G. Zhang, J. Wang, J. Chen, W. Zhou, K. Wang, and S. Yan, "AgenTracer: Who Is Inducing Failure in the LLM Agentic Systems?" in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2026, pp. 1–18.

[35] H. Zhang, Y. Shi, X. Gu, H. You, Z. Zhang, L. Gan, Y. Yuan, and J. Huang, "Graphtracer: Graph-guided failure tracing in llm agents for robust multi-turn deep search," 2025. [Online]. Available: https://arxiv.org/abs/2510.10581

[36] F. Kong, R. Zhang, H. Yin, G. Zhang, X. Zhang, Z. Chen, Z. Zhang, X. Zhang, S.-C. Zhu, and X. Feng, "Aegis: Automated Error Generation and Attribution for Multi-Agent Systems," 2025. [Online]. Available: https://arxiv.org/abs/2509.14295

[37] M. Ma, J. Zhang, F. Yang, Y. Kang, Q. Lin, T. Yang, S. Rajmohan, and D. Zhang, "DoVer: Intervention-Driven Auto Debugging for LLM Multi-Agent Systems," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2026, pp. 1–16.

[38] M. Chen, J. Wang, F. Mu, Y. Wang, Z. Liu, H. Feng, and Q. Wang, "Traceelephant: Seeing the whole elephant for failure attribution in llm-based multi-agent systems," https://github.com/TraceElephant/TraceElephant, 2026.

[39] W. Wang, P. Kattakinda, and S. Feizi, "Maestro: Joint Graph & Config Optimization for Reliable AI Agents," 2025. [Online]. Available: https://arxiv.org/abs/2509.04642

[40] J. Zhang, K. Cai, Q. Zeng, N. Liu, S. Fan, Z. Chen, and K. Wang, "Failure-Driven Workflow Refinement," 2025. [Online]. Available: https://arxiv.org/abs/2510.10035

[41] R. Costa, "Instruction-Level Weight Shaping: A Framework for Self-Improving AI Agents," 2025. [Online]. Available: https://arxiv.org/abs/2509.00251

[42] Z. Pei, H.-L. Zhen, S. Kai, S. J. Pan, Y. Wang, M. Yuan, and B. Yu, "SCOPE: Prompt Evolution for Enhancing Agent Effectiveness," 2025. [Online]. Available: https://arxiv.org/abs/2512.15374

[43] D. Zhang, "AgentDevel: Reframing Self-Evolving LLM Agents as Release Engineering," 2026. [Online]. Available: https://arxiv.org/abs/2601.04620

[44] Z. Hao, H. Wang, J. Luo, J. Zhang, Y. Zhou, Q. Lin, C. Wang, H. Dong, and J. Chen, "Recreate: Reasoning and creating domain agents driven by experience," 2026. [Online]. Available: https://arxiv.org/abs/2601.11100

[45] Y.-A. Xiao, P. Gao, C. Peng, and Y. Xiong, "Improving the Efficiency of LLM Agent Systems through Trajectory Reduction," 2025. [Online]. Available: https://arxiv.org/abs/2509.23586

[46] F. Lin, S. Chen, R. Fang, H. Wang, and T. Lin, "Stop Wasting Your Tokens: Towards Efficient Runtime Multi-Agent Systems," 2025. [Online]. Available: https://arxiv.org/abs/2510.26585

[47] Y. Zheng, Y. Hu, T. Yu, and A. Quinn, "AgentSight: System-Level Observability for AI Agents Using eBPF," in *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*, ser. SOSP '25. ACM, Oct. 2025, pp. 110–115. [Online]. Available: http://dx.doi.org/10.1145/3766882.3767169

[48] D. Moshkovich and S. Zeltyn, "Taming uncertainty via automation: Observing, analyzing, and optimizing agentic ai systems," 2025. [Online]. Available: https://arxiv.org/abs/2507.11277

[49] T. Ou, W. Guo, A. Gandhi, G. Neubig, and X. Yue, "AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, ser. EMNLP'25, 2025, pp. 1–8.

[50] M. Desmond, J. Y. Lee, I. Ibrahim, J. M. Johnson, A. Sil, J. MacNair, and R. Puri, "Agent Trajectory Explorer: Visualizing and Providing Feedback on Agent Trajectories," in *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, ser. AAAI'25/IAAI'25/EAAI'25. AAAI Press, 2025. [Online]. Available: https://doi.org/10.1609/aaai.v39i28.35350

[51] W. Epperson, G. Bansal, V. C. Dibia, A. Fourney, J. Gerrits, E. E. Zhu, and S. Amershi, "Interactive Debugging and Steering of Multi-Agent AI Systems," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI '25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: https://doi.org/10.1145/3706598.3713581

[52] X. Wang, M. Yin, E. Koh, and M. D. Dogan, "Xagen: An Explainability Tool for Identifying and Correcting Failures in Multi-Agent Workflows," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025, pp. 1–12.

[53] R. Sheng, Y. Yang, C. Shi, Y. Lin, Z. Chen, H. Qu, and F. Cheng, "Dills: Interactive diagnosis of llm-based multi-agent systems via layered summary of agent behaviors," in *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, ser. CHI '26. New York, NY, USA: Association for Computing Machinery, 2026.

[54] J. Roig, "How Do LLMs Fail In Agentic Scenarios? A Qualitative Analysis of Success and Failure Scenarios of Various LLMs in Agentic Simulations," in *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS)*, 2025, pp. 1–15.

[55] T. Ma, Y. Chen, V. Anand, A. Cornacchia, A. R. Faustino, G. Liu, S. Zhang, H. Luo, S. A. Fahmy, Z. A. Qazi, and M. Canini, "MAESTRO: Multi-Agent Evaluation Suite for Testing, Reliability, and Observability," 2026. [Online]. Available: https://arxiv.org/abs/2601.00481

[56] L. Advani, "Trajectory guard a lightweight, sequence-aware model for real-time anomaly detection in agentic AI," in *AAAI 2026 Workshop on Trust and Control in Agentic AI (TrustAgent)*, 2026, to appear.

[57] S. Chaduvula, J. Ho, K. Kim, A. Narayanan, M. Alinoori, M. Garg, D. Ramachandram, and S. Raza, "From features to actions: Explainability in traditional and agentic ai systems," 2026. [Online]. Available: https://arxiv.org/abs/2602.06841

[58] C. Han, X. Liu, H. Wang, S. Li, J. Yang, H. Jiang, Z. Wang, Q. Yin, L. Qiu, C. Yu, Y. Gao, Z. Li, B. Yin, J. Shang, and H. Ji, "Can language models follow multiple turns of entangled instructions?" in *Findings of the Association for Computational Linguistics: EMNLP 2025*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 25 445–25 460. [Online]. Available: https://aclanthology.org/2025.findings-emnlp.1387/

[59] Z. Zeng, J. Yu, T. Gao, Y. Meng, T. Goyal, and D. Chen, "Evaluating large language models at evaluating instruction following," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=tr0KidwPLc

[60] F. Elberzhager, J. Münch, and V. T. N. Nha, "A systematic mapping study on the combination of static and dynamic quality assurance techniques," *Information and software technology*, vol. 54, no. 1, pp. 1–15, 2012.

[61] R. E. Fairley, "Tutorial: Static analysis and dynamic testing of computer software," *Computer*, vol. 11, no. 4, pp. 14–23, 2006.

[62] S. B. Hossain, N. Jiang, Q. Zhou, X. Li, W.-H. Chiang, Y. Lyu, H. Nguyen, and O. Tripp, "A deep dive into large language models for automated bug localization and repair," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1471–1493, 2024.

[63] AgentOps, Inc., "AgentOps TypeScript SDK," Node Package Manager (NPM), Jun. 2025, version 0.1.0. [Online]. Available: https://www.npmjs.com/package/agentops

[64] IBM Cloud Education, "Agentops: Monitor and govern ai agents with IBM telemetry by using watsonx orchestrate," IBM Think Tutorial, 6 2025, accessed: 2026-02-13. [Online]. Available: https://www.ibm.com/think/tutorials/agentops-telemetry-watsonx-orchestrate

[65] Z. Wang, J. Li, Q. Zhou, H. Si, Y. Liu, J. Li, G. Xie, F. Sun, D. Pei, and C. Pei, "A survey on agentops: Categorization, challenges, and future directions," *CoRR*, vol. abs/2508.02121, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2508.02121

[66] Fiddler AI Team, "The production-ready agent: A practical playbook," Fiddler AI Blog, Aug. 2025, covers real-world deployment challenges including architectural trade-offs, silent failures, and the need for continuous observability in production environments. [Online]. Available: https://www.fiddler.ai/blog/production-ready-agent-playbook

[67] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds., 2024. [Online]. Available: http://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html

[68] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, M. Christakis and M. Pradel, Eds. ACM, 2024, pp. 1592–1604. [Online]. Available: https://doi.org/10.1145/3650212.3680384

[69] C. Zhang, S. He, J. Qian, B. Li, L. Li, S. Qin, Y. Kang, M. Ma, G. Liu, Q. Lin *et al.*, "Large language model-brained gui agents: A survey," *arXiv preprint arXiv:2411.18279*, 2024.

[70] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, to appear. [Online]. Available: https://arxiv.org/abs/2307.07924

[71] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for A multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=VtmBAGCN7o

[72] F. Mu, J. Wang, L. Shi, S. Wang, S. Li, and Q. Wang, "EXPEREPAIR: dual-memory enhanced llm-based repository-level program repair," *CoRR*, vol. abs/2506.10484, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2506.10484

[73] Z. Yuan, M. Liu, S. Ding, K. Wang, Y. Chen, X. Peng, and Y. Lou, "Evaluating and improving chatgpt for unit test generation," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1703–1726, 2024.

[74] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring llm-based general bug reproduction," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023, pp. 2312–2323. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00194

[75] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, "Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 2024, pp. 100:1–100:13. [Online]. Available: https://doi.org/10.1145/3597503.3639180

[76] L. Dong, Q. Lu, and L. Zhu, "Agentops: Enabling observability of llm agents," 2024. [Online]. Available: https://arxiv.org/abs/2411.05285